Introduction to the RSPH High Performance Computing (HPC) Cluster

BIOS 731 Guest Lecture

Keven Haynes, Director of Research and Platform Computing, RSPH
March 26th 2026

Why HPC?

Our personal computers have serious limitations.

- "Big Data" computations require more capability than our individual laptops or workstations can handle
- Data sets are too large for both memory and local disk
- Personal computers are running other applications, like web-browsers, iTunes, Fortnite
- Wireless connectivity is spotty and slow
- Commercial desktop operating systems are not optimized for computation or performance

What is HPC?

Any number of methods for solving computational problems using dedicated computers, usually in parallel.

- Parallelization (i.e., running multiple instances of computation at the same time) across multiple computers allows for (sometimes *linear*) "speedup"
- We aggregate large numbers of computers that have more processors and very large installations of RAM
- Computers are connected via high-speed, reliable, wired networks and use specialized "parallel" file systems for availability and performance.

Our HPC Cluster

- 161 “compute nodes” for an aggregate number of 5152 compute cores
- 196/256 GB of RAM per node (“largemem” nodes have ~1.5TB of RAM)
- Individual 25Gb ethernet (wired) connection
- Access to 1.2 Petabyte parallel filesystem (“PanFS”, by VDura)
- All nodes run the Linux Operating System (Rocky 8, like RHEL 8).
- Designed for serial/batch operation* (as opposed to interactive use) using the **SLURM** job scheduler

Who uses the RSPH HPC cluster

- There are currently over 400 active accounts on the RSPH HPC cluster. They belong to Rollins faculty, students and staff, as well as collaborators from other schools at Emory and other institutions
- There are 40+ active research groups using the HPC cluster
- There are typically around 25-35 active users on the cluster during business hours.
- On average, the cluster is running over 1000 concurrent jobs
- There are two+ HPC staff members (reorg underway)

Who *Should* Use the HPC Cluster?

Some general guidelines for work appropriate for an HPC environment:

- Users with really large data sets that require large amounts of computation, for starters. (Jobs or programs that can complete during a single “sitting” are probably more appropriate for a workstation.)
- Users with some programming expertise, or who are working within a group providing access to necessary expertise.
- Users with Linux knowledge, or the ability/desire to learn Linux.
- Jobs/programs that can operate in an non-GUI, command line, serial/batch environment. Interactive work is the exception, not the rule, on the HPC cluster.

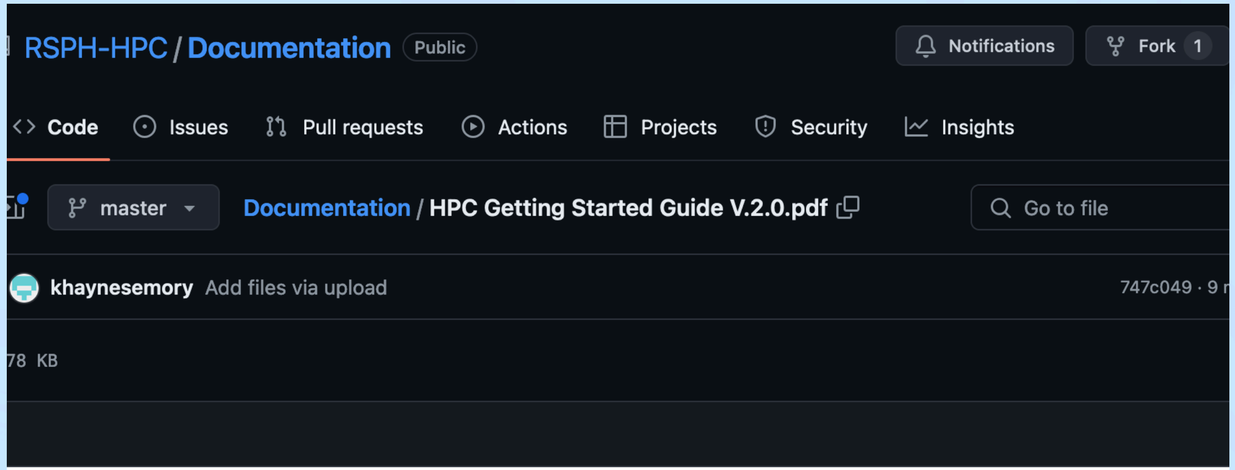
Accessing the HPC Cluster

- Get an account via RSPH Faculty sponsorship. Faculty must request user accounts on recipients' behalf via a Help Request Ticket.
- Read the Welcome Email (please).
- Download the RSPH "High Performance Computing Getting Started Guide v 2.0". Read it.
- Do the VPN "handshake". Once your VPN client is working, request the privilege escalation so that your account can access the Cluster's network.
- Use your local computer's **ssh** client (every computer has one via the terminal) to login to the cluster. You do not need another client, such as Putty, FileZilla, MobaXterm, etc.

The HPC Getting Started Guide V.2.0

Available at
https://github.com/RSPH-HPC/Documentation

- A link to the Guide is included in the Welcome Email.
- There are 30 pages of information including example code
- It contains important cluster policy descriptions that all users implicitly agree to abide by.

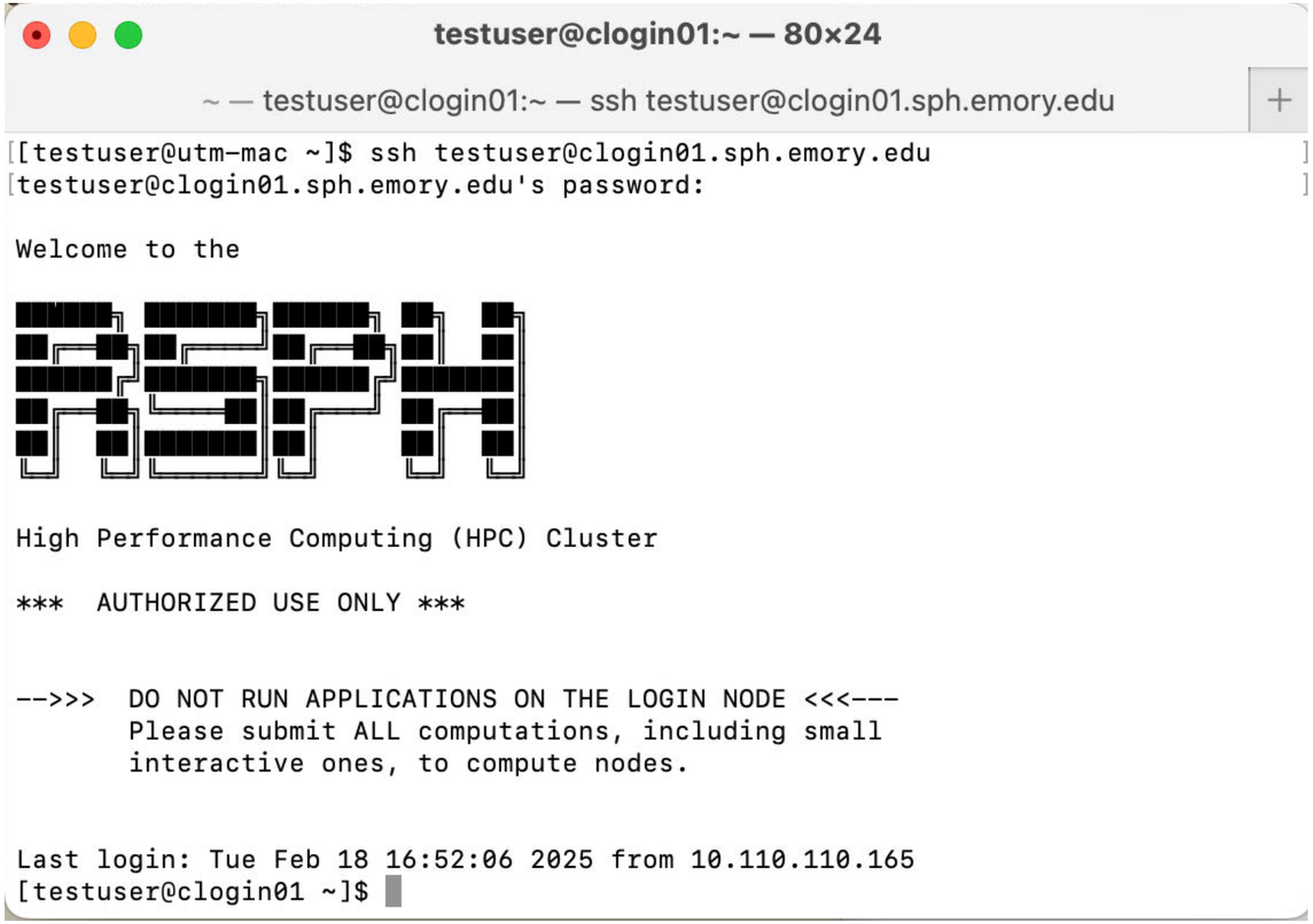


The Emory Rollins School of Public Health

High Performance Computing Getting Started Guide

Logging Into the Cluster

- Open your computer's Terminal application
- Type: `ssh <userID>@clogin01.sph.emory.edu`
- Enter the password provided in the Welcome Email.
- You should be prompted to change your password immediately. When you are prompted, enter your starter password first, followed by your new one.



Changing Your Password Manually

- Use the **`passwd`** command
- Enter your current password first, then your new password twice
- Your password will not be echoed (i.e., displayed) on the screen as you type. This is a security feature.

Getting Started - Data Transfer

- The main way to move data to the cluster is via **`scp`** or **`sftp`**, which can be executed from your local computer or laptop. It is a part of OpenSSH and should be available by default in new Windows systems.
- Data can be "pulled" to the HPC Cluster. Pulling directly from external sites (instead of downloading to your local computer first) avoids the slowness of the Emory VPN. Use applications such as **`wget, curl, rsync`** and **`scp/sftp`** from the command line on the cluster.
- The **`git`** client is also available for pulling and pushing to git repositories from the cluster.
- The AWS-CLI is available as a cluster module for connecting to S3/Glacier buckets.
- One can also install and use the Globus Personal Connect client to connect to Globus endpoints. Users can install the client individually.

Getting Started - Data Advice

- *Make backup copies of all otherwise irreplaceable data.* ***No backups are made of any user data on the HPC cluster.*** Large data repositories from published sites don't require backing up if they are accessible elsewhere on the internet. Your source code (which you may have authored, for instance), may be unique - using a version control system like **`git`** can prevent disasters.

- *Consider your data's organization up front and early in a project.* If you are required to save large amounts of data for documentation purposes, consider an AWS bucket (like S3 or Glacier) for cluster-accessible data storage (and backups!). AWS is superior to OneDrive and the R:\ drive because of its Linux compatibility, reliability and portability.

- *Don't "index" your files using long, informative names.* Long names are useful if the number of files is small, but they make management very difficult if you have hundreds of thousands of them. The data structure of the file system may not be able to hold all the names in memory on a system during, say, an **`ls`**, making large data operations impossible.

- *Don't use special characters and spaces in your file names.* It's not illegal, it just makes data management harder. (No emojis!)

- **The HPC staff does not do "data curation"**. Management of files, file permissions and ownership that do not require root/admin access will be left to the end-user/file owner.

No backups of any data are performed on the HPC Cluster.

Learn the Linux Command Line!

- Some basic vital Linux commands: **`ls, pwd, cd, mv, cp, rm`** and... **`exit`**

- I recommend the use of the flags **`-iv`** for move, copy and remove. **`-v`** is for *verbose* and **`-i`** is for *interactive*, which means it will ask you if you intend to "clobber" a file. These options can help prevent or help you recover from mistakes.

- One of the most important and underused commands is **`man`**, which stands for *manual*. Our Linux distribution comes with a very complete and verbose manual covering nearly every aspect of Linux usage, all available from the command line. (Exercise: type **`man`** `<command>` for each of the above listed commands.)

- An entire course could be spent on Linux. I recommend most books from OReilly on the the subject, including their interactive videos. LinkedIn Learning also has very thorough introduction to the Linux command line. A popular free online book on the subject is *The Linux Command Line* (https://www.linuxcommand.org/tlcl.php)

Editing Files

Too many choices…

- For large coding projects, you probably want to work in an IDE on your local system, especially if you have a desired preference. You then upload a completed file and then run a small-version test.
- You can also edit interactively on the cluster using command line tools. They are harder to learn, but are very useful and standard on any Linux system.
- **`Cat`** and **`more`**/**`less`** can display a text file. **`Vim`**, **`emacs`** and **`nano`** are text line editors.

```
[testuser@clogin01 ~]$ cat HelloWorld.py
#! /apps/Python-3.11.3/bin/python3
import socket

hostname = socket.gethostname()

print("\nHello World from " + hostname + "!\n")


[testuser@clogin01 ~]$ more HelloWorld.py
#! /apps/Python-3.11.3/bin/python3
import socket

hostname = socket.gethostname()

print("\nHello World from " + hostname + "!\n")


[testuser@clogin01 ~]$
```

```
#! /apps/Python-3.11.3/bin/python3
import socket

hostname = socket.gethostname()

print("\nHello World from " + hostname + "!\n")
~
~
~
~
~
~
~
~
~
~
~
~
~
~
"HelloWorld.py" 6L, 131C
```

```
File Edit Options Buffers Tools Python Help
#! /apps/Python-3.11.3/bin/python3
import socket

hostname = socket.gethostname()

print("\nHello World from " + hostname + "!\n")

-UU-:----F1  HelloWorld.py    All L1     (Python ElDoc) ---
Can't guess python-indent-offset, using defaults: 4
```

```
  GNU nano 2.9.8                          HelloWorld.py

#! /apps/Python-3.11.3/bin/python3
import socket

hostname = socket.gethostname()

print("\nHello World from " + hostname + "!\n")

                                   [ Read 6 lines ]
^G Get Help    ^O Write Out   ^W Where Is    ^K Cut Text    ^
^X Exit        ^R Read File   ^\ Replace     ^U Uncut Text  ^
```

Running Jobs

- Computations on the cluster are submitted for scheduling in the form of a "job".
- Jobs are submitted by calling the **`sbatch`** command followed by the program name in the simplest case, for example, to run submit a Python script called HelloWorld.py, we'd type **`sbatch HelloWorld.py`**. Entering that command yields a return statement with the submitted jobs **`jobid`**. (See next slide for picture.)
- We can monitor the jobs we submit with the **`squeue`** command. To limit the output to our own jobs, we add the **`-u`** flag with our userid. (See example slide.)
- To cancel any submitted jobs, we use the **`scancel`** command with the **`jobid`** of the job we wish to cancel. To cancel all submitted jobs by a user, we can again use the **`-u`** flag with our userid to cancel all of our jobs simultaneously.
- The **`squeue`** command identifies the state (whether running, "**`R`**", or pending "**`PD`**") and what compute node it has been assigned to, or the reason why it has not yet been assigned to a node. In this example, all of the submitted jobs are in a pending state due to "**`Priority`**", meaning that another running job has priority over this submission (whether due to partition or simply arriving later in the queue).

Running a program versus Submitting a "job"

Job submissions are run on compute nodes and yield output files.

```
[testuser@clogin01 Testdir]$ cat /home/testuser/HelloWorld.py
#! /apps/Python-3.11.3/bin/python3
import socket

hostname = socket.gethostname()

print("\nHello World from " + hostname + "!\n")


[testuser@clogin01 Testdir]$ /home/testuser/HelloWorld.py

Hello World from clogin01!

[testuser@clogin01 Testdir]$ ls
[testuser@clogin01 Testdir]$ sbatch /home/testuser/HelloWorld.py
Submitted batch job 37742847
[testuser@clogin01 Testdir]$ ls
slurm-37742847.out
[testuser@clogin01 Testdir]$ cat slurm-37742847.out

Hello World from node9!

[testuser@clogin01 Testdir]$
```

sbatch, squeue and scancel

```
[[testuser@clogin01 ~]$ sbatch HelloWorld.py
 Submitted batch job 26953615
[[testuser@clogin01 ~]$ squeue -u testuser
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
          26953615 short-cpu HelloWor testuser PD       0:00      1 (Priority)
[[testuser@clogin01 ~]$ sbatch HelloWorld.py
 Submitted batch job 26953616
[[testuser@clogin01 ~]$ sbatch HelloWorld.py
 Submitted batch job 26953617
[[testuser@clogin01 ~]$ squeue -u testuser
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
          26953615 short-cpu HelloWor testuser PD       0:00      1 (Priority)
          26953616 short-cpu HelloWor testuser PD       0:00      1 (Priority)
          26953617 short-cpu HelloWor testuser PD       0:00      1 (Priority)
[[testuser@clogin01 ~]$ scancel -u testuser
[[testuser@clogin01 ~]$ squeue -u testuser
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
[[testuser@clogin01 ~]$ sbatch HelloWorld.py
 Submitted batch job 26953733
[[testuser@clogin01 ~]$ sbatch HelloWorld.py
 Submitted batch job 26953734
[[testuser@clogin01 ~]$ sbatch HelloWorld.py
 Submitted batch job 26953735
[[testuser@clogin01 ~]$ squeue -u testuser
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
          26953733 short-cpu HelloWor testuser PD       0:00      1 (Priority)
          26953734 short-cpu HelloWor testuser PD       0:00      1 (Priority)
          26953735 short-cpu HelloWor testuser PD       0:00      1 (Priority)
[[testuser@clogin01 ~]$ scancel  26953734
[[testuser@clogin01 ~]$ squeue -u testuser
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
          26953733 short-cpu HelloWor testuser PD       0:00      1 (Priority)
          26953735 short-cpu HelloWor testuser PD       0:00      1 (Priority)
 [testuser@clogin01 ~]$
```

Running Jobs: Batch files

- To aid in job submission, one can create a *batch file*, or job-submission file. A batch file is a script written in the **`bash`** syntax that contains SLURM processing directives. These directives can be used to set a very large number of options for submitting jobs, such as the name of the job(s), the names of output and error files, an email address for notifying the user about changes in job state, the partitions and/or nodes requested, and, importantly, the resources, such as the number of cores and RAM, can also be requested.

- Other features, such as the loading of modules and the use of *array parameters* can be established within a batch file.

- SLURM *job arrays* permit one to submit a large number of similar jobs efficiently without the use of programmed for-loops, for instance. One can use the variable **`$SLURM_ARRAY_TASK_ID`** to refer to specific elements (job instances) within the submitted job array. This is the most efficient way of submitting very large repetitive tasks on the HPC cluster.

- The HPC cluster is limited to handling 25000 job submissions at any single point in time. Users are similarly asked to limit any submissions to 2000 jobs at any one time. Users who submit too many jobs as once may receive angry emails from other users.

Example Batch File with Job Array

- Using the **`sinfo`** command, we can view the available partitions on the cluster

- Using the Linux command **`more`**, we can view our job-submission file. This example submits 11 copies of the script **`normal.R`** to run on the **`day-long-cpu`** partition. Each job will write its output to their separate file with the job id and array id in its name (represented by %A and %a respectively).

```
[testuser@clogin01 ~]$ sinfo
PARTITION        AVAIL  TIMELIMIT  NODES  STATE NODELIST
month-long-cpu      up 31-00:00:0     22  alloc node[1-22]
week-long-cpu       up 7-00:00:00     20  alloc node[1-20]
day-long-cpu        up 1-00:00:00     20  alloc node[3-22]
short-cpu*          up    8:00:00     22  alloc node[1-22]
interactive-cpu     up   12:00:00      2    mix node[23-24]
largemem            up 7-01:00:00      1    mix node25
largemem            up 7-01:00:00      2   idle node[92-93]
[testuser@clogin01 ~]$ more normal.sh
#!/bin/bash

#SBATCH --job-name=normal.R
#SBATCH --partition=day-long-cpu
#SBATCH --mail-type=END,FAIL
#SBATCH --mail-user=khaynes@emory.edu
#SBATCH --output=Out/normal.%A_%a.out
#SBATCH --error=Err/normal.%A_%a.err
#SBATCH --array=0-10

module purge
module load R

srun normal.R
[testuser@clogin01 ~]$
```

Sbatch and for-loops: BAD

- Do not submit jobs to SLURM from within a for-loop. This is very taxing on the scheduler. Use job arrays instead.
- Eliminate unnecessary file I/O from job submissions. Perform file and directory creations before hand.
- Avoid “race conditions” - writing to a file, and immediately deleting it, perhaps before SLURM has read it!

```
#!/bin/bash
for item in 10
do

for number in 5 10 20
do

for rep in {1..1000}
do

echo "#!/bin/bash" >> my_sim_batch.sh
echo "#SBATCH -J my_simulation" >> my_sim_batch.sh
echo "#SBATCH --partition=fake-cpu" >> my_sim_batch.sh
echo "#SBATCH --output /projects/testuser/research/my_sim_%J.out" >> my_sim_batch.sh
echo "#SBATCH --error /projects/testuser/research/my_sim_%J.err" >> my_sim_batch.sh
echo "module purge" >> my_sim_batch.sh
echo "module load python3" >> my_sim_batch.sh

echo "srun python /projects/testuser/research/my_sim.py --item=${item} --number=${number} -
-rep=${rep}" >> my_sim_batch.sh

#### Unneccessary I/O!!!
chmod +x my_sim_batch.sh
qsub ./my_sim_batch.sh    # use sbatch!!
rm -rf my_sim_batch.sh    # NO!!!!!!

done
done
done
```

Advanced Job Array Example

- This example uses the **`$SLURM_ARRAY_TASK_ID`** to identify which row of parameters from a file to use as inputs to a script.
- Use of job arrays in this way can completely remove the need for for-loop structures in a job batch script.
- One should never call **`sbatch`** from within a for-loop or another looping program, as it strains the scheduler.

```
#!/bin/bash
#SBATCH --job-name=advancedArray
#SBATCH --output=advancedArray.%A_%a.out
#SBATCH --error=advancedArray.%A_%a.err
#SBATCH --partition=short-cpu
#SBATCH --array=1-100

# This method imports parameters from the file 'input_file.csv'
# whose rows are of the form:
#
# $head input_file.csv
#  1,300,2,FALSE
#  2,300,2,TRUE
#  3,150,2,FALSE
#  4,150,2,TRUE
#    ...
#

input_list=$( awk "NR==$SLURM_ARRAY_TASK_ID" input_file.csv )
seed=$( echo "$input_list" | cut -d "," -f 1 )
size=$( echo "$input_list" | cut -d "," -f 2 )
type=$( echo "$input_list" | cut -d "," -f 3 )
bias=$( echo "$input_list" | cut -d "," -f 4 )

module purge
module load R


Rscript advancedArrayEx.R $seed $size $type $bias
```

Programming Advice

Smaller and simpler are guiding principles.

- Keeping tasks small and simple (atomic) makes it easier for the scheduler to find job slots. This is not always possible, of course (e.g. large memory programs). Jobs assigned to the shorter-duration (general use) partitions run at a higher priority.

- Let the scheduler, SLURM, do the parallelization, if possible. Parallel code designed on a laptop may not scale well to the HPC cluster. Sometimes it is better overall to program for serial computations and then run jobs in parallel on the cluster.

- Always make sure that you account for the number of cpus/cores you intend to use in SLURM. Jobs that oversubscribe nodes will be killed (either by the admins or by the operating system itself).

Policies

- Do not run applications on the login node. All computations must be submitted to a compute node via SLURM.
- Users may not run any sort of server or service on the HPC cluster. This includes, but is not limited to, **VScode-server**, **PyCharm server**, or a network tunnel. Failure to comply may result in loss of access. (You may run Jupyter notebooks from a compute node - see the Getting Started Guide.)
- Do not share your HPC cluster credentials with anyone.
- The cluster is a shared system. You are responsible for protecting your data from unwanted/ illegal access. Make sure that group ownership of files and permissions are set correctly.
- Be a good neighbor! Don’t engage in activities that result in a loss of service for others.

Getting Help and Additional Resources

- To open an official Help Request from the HPC staff, email help@sph.emory.edu with the subject line "**RSPH-HPC: <nature_of_request_or_issue>**". Please include your Emory NetID (i.e., your cluster userID) in the body of the request, and try to offer some details about the request (in other words, don't just say "something's broken" - be specific, include error messages, etc).

- Carbon-copy me (i.e., CC: keven.haynes@emory.edu) on the request to make sure I receive it. HPC tickets that don't specify RSPH sometimes go to OIT and take days to get re-routed!

- Extra SLURM help: https://slurm.schedmd.com/quickstart.html

- Other academic HPC sites sometimes have good suggestions. If you ever have questions, feel free to run them by the HPC staff before launching code.